

TP - ANDROID - Lab 7 : Persistance locale des données avec Room

1. Objectifs pédagogiques

À la fin de ce TP, l'étudiant sera capable de :

- Comprendre le rôle de Room dans une application Android
- Transformer une classe Java en entité Room avec `@Entity`
- Créer un DAO avec `@Insert`, `@Update`, `@Delete` et `@Query`
- Créer une base locale Room avec `RoomDatabase`
- Utiliser un singleton thread-safe pour accéder à la base
- Exécuter les opérations Room dans un thread secondaire avec `ExecutorService`
- Revenir sur le thread UI avec `runOnUiThread()`
- Remplacer une liste statique par des données persistées localement
- Persister les ajouts, modifications et suppressions de livres

2. Prérequis techniques

Room est la bibliothèque Android recommandée pour travailler avec SQLite de manière plus simple et plus sûre. Elle repose sur trois éléments principaux : `Entity`, `DAO` et `Database`. La version stable actuelle de Room 2.x indiquée par la documentation Android est **2.8.4**. Pour un projet Java, on utilise `annotationProcessor` avec `room-compiler`.

Dépendances avec Version Catalog

Ouvrir :

```
1 | gradle/libs.versions.toml
```

Ajouter :

```
1 | [versions]
2 | room = "2.8.4"
3 |
4 | [libraries]
5 | room-runtime = { module = "androidx.room:room-runtime", version.ref =
6 | "room" }
7 | room-compiler = { module = "androidx.room:room-compiler", version.ref =
8 | "room" }
```

Ensuite, dans le fichier :

```
1 | app/build.gradle
```

Ajouter :

```

1 dependencies {
2     implementation(libs.appcompat)
3     implementation(libs.material)
4     implementation(libs.activity)
5     implementation(libs.constraintlayout)
6     implementation(libs.recyclerview)
7     implementation(libs.cardview)
8
9     implementation(libs.room.runtime)
10    annotationProcessor(libs.room.compiler)
11
12    testImplementation(libs.junit)
13    androidTestImplementation(libs.ext.junit)
14    androidTestImplementation(libs.espresso.core)
15 }

```

Après modification :

- cliquer sur **Sync Now**
- attendre la fin de la synchronisation Gradle

3. Mise en situation

Dans les labs précédents, la liste des livres était stockée dans une `ArrayList`.

Problème : lorsque l'application est fermée, les livres ajoutés ou modifiés disparaissent.

La bibliothèque municipale souhaite maintenant que les livres soient conservés dans le téléphone.

Dans ce lab, vous allez remplacer la liste statique par une vraie base de données locale avec **Room**.

4. Étapes de réalisation

Étape 1 : Comprendre Room

Room est une couche au-dessus de SQLite.

Elle permet de manipuler une base de données locale avec du code Java plus simple.

Room repose principalement sur trois concepts :

- `@Entity` : représente une table
- `@Dao` : contient les opérations SQL
- `@Database` : représente la base de données

Dans notre application :

- `Livre.java` devient une table

- `LivreDao.java` contient les requêtes
- `AppDatabase.java` donne accès à la base

Étape 2 : Transformer `Livre.java` en entité Room

Ouvrir :

1 `Livre.java`

Remplacer par :

```
1 import androidx.room.Entity;
2 import androidx.room.PrimaryKey;
3
4 import java.io.Serializable;
5
6 // @Entity indique à Room que cette classe représente une table
7 // SQLite.
8 // tableName permet de donner un nom clair à la table.
9 @Entity(tableName = "livres")
10 public class Livre implements Serializable {
11     // @PrimaryKey indique la clé primaire de la table.
12     // autoGenerate = true permet à Room de générer automatiquement
13     // l'id.
14     @PrimaryKey(autoGenerate = true)
15     private int id;
16
17     private String titre;
18     private String auteur;
19     private String isbn;
20     private boolean disponible;
21
22     public Livre(int id, String titre, String auteur, String isbn,
23 boolean disponible) {
24         this.id = id;
25         this.titre = titre;
26         this.auteur = auteur;
27         this.isbn = isbn;
28         this.disponible = disponible;
29     }
30
31     public int getId() {
32         return id;
33     }
34
35     public void setId(int id) {
36         this.id = id;
37     }
38
39     public String getTitre() {
40         return titre;
41     }
42
43     public void setTitre(String titre) {
44         this.titre = titre;
45     }
46
47     public String getAuteur() {
48         return auteur;
```

```

49     }
50
51     public void setAuteur(String auteur) {
52         this.auteur = auteur;
53     }
54
55
56     public String getIsbn() {
57         return isbn;
58     }
59
60     public void setIsbn(String isbn) {
61         this.isbn = isbn;
62     }
63
64
65     public boolean isDisponible() {
66         return disponible;
67     }
68
69     public void setDisponible(boolean disponible) {
70         this.disponible = disponible;
71     }
72 }

```

Explication

Avant, `Livre` était seulement une classe Java.

Maintenant, avec `@Entity`, elle devient une table dans la base locale.

Chaque objet `Livre` correspondra à une ligne dans la table `livres`.

Étape 3 : Créer `LivreDao.java`

Créer une nouvelle interface Java :

```
1 | LivreDao.java
```

Ajouter :

```

1  import androidx.room.Dao;
2  import androidx.room.Delete;
3  import androidx.room.Insert;
4  import androidx.room.Query;
5  import androidx.room.Update;
6
7  import java.util.List;
8
9  // @Dao indique à Room que cette interface contient les requêtes SQL.
10 @Dao
11 public interface LivreDao {
12
13     // Insère un nouveau livre dans la base.
14     @Insert
15     void insert(Livre livre);
16
17     // Modifie un livre existant.
18     @Update
19     void update(Livre livre);
20

```

```

21 // Supprime un livre existant.
22 @Delete
23 void delete(Livre livre);
24
25 // Récupère tous les livres, du plus récent au plus ancien.
26 @Query("SELECT * FROM livres ORDER BY id DESC")
27 List<Livre> getAllLivres();
28
29 // Supprime tous les livres.
30 // Utile pour réinitialiser la base pendant les tests.
31 @Query("DELETE FROM livres")
32 void deleteAll();
33 }

```

Explication

Le DAO est la porte d'entrée vers les données.

Au lieu d'écrire directement du SQL partout dans l'application, on regroupe les requêtes ici.

Étape 4 : Créer `AppDatabase.java`

Créer une nouvelle classe Java :

```
1 AppDatabase.java
```

Ajouter :

```

1 import android.content.Context;
2
3 import androidx.room.Database;
4 import androidx.room.Room;
5 import androidx.room.RoomDatabase;
6
7 // @Database indique à Room quelles entités font partie de la base.
8 // version = 1 indique la première version de notre schéma.
9 @Database(entities = {Livre.class}, version = 1, exportSchema = false)
10 public abstract class AppDatabase extends RoomDatabase {
11
12     // Instance unique de la base.
13     private static volatile AppDatabase INSTANCE;
14
15     // Méthode abstraite qui donne accès au DAO.
16     public abstract LivreDao livreDao();
17
18     // Singleton thread-safe avec double-checked locking.
19     public static AppDatabase getInstance(Context context) {
20         if (INSTANCE == null) {
21             synchronized (AppDatabase.class) {
22                 if (INSTANCE == null) {
23                     INSTANCE = Room.databaseBuilder(
24                         context.getApplicationContext(),
25                         AppDatabase.class,
26                         "bibliotheque_database"
27                     )
28                         // Pour un TP d'initiation.
29                         // En production, on utiliserait des
30                         migrations.
31                         .fallbackToDestructiveMigration(false)
32                         .build();
33                 }
34             }
35         }
36     }
37 }

```

```

34     }
35
36     return INSTANCE;
37 }
38 }

```

Explication

`AppDatabase` représente la base de données locale.

Le singleton évite de créer plusieurs instances de la base en même temps.

`volatile` et `synchronized` rendent l'accès plus sûr quand plusieurs threads peuvent utiliser la base.

Étape 5 : Préparer l'Adapter pour modifier et supprimer

Dans les labs précédents, l'Adapter ouvrait directement l'écran détail.

Nous allons lui ajouter deux callbacks :

- clic simple : ouvrir détail
- clic long : modifier ou supprimer

Ouvrir :

```
1 LivreAdapter.java
```

Remplacer par :

```

1  import android.graphics.Color;
2  import android.view.LayoutInflater;
3  import android.view.View;
4  import android.view.ViewGroup;
5  import android.widget.TextView;
6  import android.widget.Toast;
7
8  import androidx.annotation.NonNull;
9  import androidx.recyclerview.widget.RecyclerView;
10
11 import java.util.List;
12
13 public class LivreAdapter extends
14     RecyclerView.Adapter<LivreAdapter.LivreViewHolder> {
15
16     public interface OnLivreClickListener {
17         void onLivreClick(Livre livre);
18         void onLivreLongClick(Livre livre, int position);
19     }
20
21     private List<Livre> listeLivres;
22     private OnLivreClickListener listener;
23
24     public LivreAdapter(List<Livre> listeLivres, OnLivreClickListener
25         listener) {
26         this.listeLivres = listeLivres;
27         this.listener = listener;
28     }

```

```

27
28     @NonNull
29     @Override
30     public LivreViewHolder onCreateViewHolder(@NonNull ViewGroup
parent, int viewType) {
31         View view = LayoutInflater.from(parent.getContext())
32             .inflate(R.layout.item_livre, parent, false);
33
34         return new LivreViewHolder(view);
35     }
36
37     @Override
38     public void onBindViewHolder(@NonNull LivreViewHolder holder, int
position) {
39         Livre livre = listeLivres.get(position);
40
41         holder.tvTitreLivre.setText(livre.getTitre());
42         holder.tvAuteurLivre.setText("Auteur : " + livre.getAuteur());
43         holder.tvIsbnLivre.setText("ISBN : " + livre.getIsbn());
44
45         if (livre.isDisponible()) {
46             holder.tvDisponibilite.setText("Disponible");
47
48             holder.tvDisponibilite.setBackgroundColor(Color.parseColor("#2E7D32"))
;
49         } else {
50             holder.tvDisponibilite.setText("Indisponible");
51
52             holder.tvDisponibilite.setBackgroundColor(Color.parseColor("#C62828"))
;
53         }
54
55         holder.itemView.setOnClickListener(v -> {
56             if (listener != null) {
57                 listener.onLivreClick(livre);
58             }
59         });
60
61         holder.itemView.setOnLongClickListener(v -> {
62             if (listener != null) {
63                 int currentPosition = holder.getAdapterPosition();
64
65                 if (currentPosition != RecyclerView.NO_POSITION) {
66                     listener.onLivreLongClick(livre, currentPosition);
67                 }
68             }
69
70             return true;
71         });
72     }
73
74     @Override
75     public int getItemCount() {
76         return listeLivres.size();
77     }
78
79     public static class LivreViewHolder extends
RecyclerView.ViewHolder {
80
81         TextView tvTitreLivre;
82         TextView tvAuteurLivre;
83         TextView tvIsbnLivre;
84         TextView tvDisponibilite;
85
86         public LivreViewHolder(@NonNull View itemView) {
87             super(itemView);
88         }
89     }

```

```

87         tvTitreLivre = itemView.findViewById(R.id.tvTitreLivre);
88         tvAuteurLivre = itemView.findViewById(R.id.tvAuteurLivre);
89         tvIsbnLivre = itemView.findViewById(R.id.tvIsbnLivre);
90         tvDisponibilite =
itemView.findViewById(R.id.tvDisponibilite);
91     }
92 }
93 }

```

Explication

L'Adapter ne doit pas tout gérer seul.

Il affiche les données et informe `MainActivity` quand un utilisateur clique sur un livre.

C'est `MainActivity` qui décidera quoi faire.

Étape 6 : Mettre à jour `MainActivity.java`

Ouvrir :

```
1 MainActivity.java
```

Remplacer par :

```

1  import android.content.DialogInterface;
2  import android.content.Intent;
3  import android.os.Bundle;
4  import android.widget.Toast;
5
6  import androidx.activity.result.ActivityResultLauncher;
7  import androidx.activity.result.contract.ActivityResultContracts;
8  import androidx.appcompat.app.AlertDialog;
9  import androidx.appcompat.app.AppCompatActivity;
10 import androidx.recyclerview.widget.LinearLayoutManager;
11 import androidx.recyclerview.widget.RecyclerView;
12
13 import
com.google.android.material.floatingactionbutton.FloatingActionButton
;
14
15 import java.util.ArrayList;
16 import java.util.List;
17 import java.util.concurrent.ExecutorService;
18 import java.util.concurrent.Executors;
19
20 public class MainActivity extends AppCompatActivity {
21
22     private RecyclerView recyclerViewLivres;
23     private FloatingActionButton fabAjouterLivre;
24
25     private LivreAdapter livreAdapter;
26     private List<Livre> listeLivres;
27
28     private AppDatabase database;
29     private ExecutorService executorService;
30
31     private ActivityResultLauncher<Intent> addEditLauncher;
32
33     @Override
34     protected void onCreate(Bundle savedInstanceState) {

```



```

35     super.onCreate(savedInstanceState);
36     setContentView(R.layout.activity_main);
37
38     // Initialisation des vues
39     recyclerViewLivres = findViewById(R.id.recyclerViewLivres);
40     fabAjouterLivre = findViewById(R.id.fabAjouterLivre);
41
42     // Initialisation de la base Room
43     database = AppDatabase.getInstance(this);
44
45     // Un seul thread pour exécuter les opérations en base
46     executorService = Executors.newSingleThreadExecutor();
47
48     listeLivres = new ArrayList<>();
49
50     livreAdapter = new LivreAdapter(listeLivres, new
LivreAdapter.OnLivreClickListener() {
51         @Override
52         public void onLivreClick(Livre livre) {
53             ouvrirDetailLivre(livre);
54         }
55
56         @Override
57         public void onLivreLongClick(Livre livre, int position) {
58             afficherOptionsLivre(livre);
59         }
60     });
61
62     recyclerViewLivres.setLayoutManager(new
LinearLayoutManager(this));
63     recyclerViewLivres.setAdapter(livreAdapter);
64
65     initialiserActivityResultLauncher();
66
67     fabAjouterLivre.setOnClickListener(v ->
ouvrirFormulaireAjout());
68
69     chargerLivresDepuisRoom();
70 }
71
72 private void initialiserActivityResultLauncher() {
73     addEditLauncher = registerForActivityResult(
74         new ActivityResultContracts.StartActivityForResult(),
75         result -> {
76             if (result.getResultCode() == RESULT_OK &&
result.getData() != null) {
77
78                 Intent data = result.getData();
79
80                 Livre livre = (Livre)
data.getSerializableExtra(AddEditActivity.EXTRA_LIVRE);
81                 String mode =
data.getStringExtra(AddEditActivity.EXTRA_MODE);
82
83                 if (livre == null) {
84                     return;
85                 }
86
87                 if (AddEditActivity.MODE_ADD.equals(mode)) {
88                     ajouterLivreDansRoom(livre);
89                 } else if
(AddEditActivity.MODE_EDIT.equals(mode)) {
90                     modifierLivreDansRoom(livre);
91                 }
92             }
93         }
94     );

```

```

95     }
96
97     private void chargerLivresDepuisRoom() {
98         executorService.execute(() -> {
99             // Cette opération se fait dans un thread secondaire.
100             List<Livre> livresDepuisBase =
101             database.livreDao().getAllLivres();
102
103             runOnUiThread(() -> {
104                 // Mise à jour de l'interface sur le thread UI.
105                 listeLivres.clear();
106                 listeLivres.addAll(livresDepuisBase);
107                 livreAdapter.notifyDataSetChanged();
108             });
109         });
110
111     private void ajouterLivreDansRoom(Livre livre) {
112         executorService.execute(() -> {
113             // L'id sera généré automatiquement par Room.
114             livre.setId(0);
115
116             database.livreDao().insert(livre);
117
118             runOnUiThread(() -> {
119                 Toast.makeText(this, "Livre ajouté",
120                 Toast.LENGTH_SHORT).show();
121                 chargerLivresDepuisRoom();
122             });
123         });
124
125     private void modifierLivreDansRoom(Livre livre) {
126         executorService.execute(() -> {
127             database.livreDao().update(livre);
128
129             runOnUiThread(() -> {
130                 Toast.makeText(this, "Livre modifié",
131                 Toast.LENGTH_SHORT).show();
132                 chargerLivresDepuisRoom();
133             });
134         });
135
136     private void supprimerLivreDansRoom(Livre livre) {
137         executorService.execute(() -> {
138             database.livreDao().delete(livre);
139
140             runOnUiThread(() -> {
141                 Toast.makeText(this, "Livre supprimé",
142                 Toast.LENGTH_SHORT).show();
143                 chargerLivresDepuisRoom();
144             });
145         });
146
147     private void ouvrirFormulaireAjout() {
148         Intent intent = new Intent(MainActivity.this,
149         AddEditActivity.class);
150         intent.putExtra(AddEditActivity.EXTRA_MODE,
151         AddEditActivity.MODE_ADD);
152
153         addEditLauncher.launch(intent);
154
155     private void ouvrirFormulaireModification(Livre livre) {

```

```

155     Intent intent = new Intent(MainActivity.this,
AddEditActivity.class);
156     intent.putExtra(AddEditActivity.EXTRA_MODE,
AddEditActivity.MODE_EDIT);
157     intent.putExtra(AddEditActivity.EXTRA_LIVRE, livre);
158
159     addEditLauncher.launch(intent);
160 }
161
162 private void ouvrirDetailLivre(Livre livre) {
163     Intent intent = new Intent(MainActivity.this,
DetailActivity.class);
164     intent.putExtra("livre", livre);
165     startActivity(intent);
166 }
167
168 private void afficherOptionsLivre(Livre livre) {
169     String[] options = {"Modifier", "Supprimer"};
170
171     AlertDialog.Builder builder = new AlertDialog.Builder(this);
172     builder.setTitle(livre.getTitre());
173     builder.setItems(options, (dialog, which) -> {
174         if (which == 0) {
175             ouvrirFormulaireModification(livre);
176         } else if (which == 1) {
177             confirmerSuppression(livre);
178         }
179     });
180
181     builder.show();
182 }
183
184 private void confirmerSuppression(Livre livre) {
185     new AlertDialog.Builder(this)
186         .setTitle("Supprimer le livre")
187         .setMessage("Voulez-vous vraiment supprimer ce livre
?")
188         .setPositiveButton("Supprimer", (dialog, which) ->
supprimerLivreDansRoom(livre))
189         .setNegativeButton("Annuler", null)
190         .show();
191 }
192
193 @Override
194 protected void onDestroy() {
195     super.onDestroy();
196
197     // On ferme proprement le thread quand l'activité est
détruite.
198     if (executorService != null) {
199         executorService.shutdown();
200     }
201 }
202 }

```

Explication

L'ancienne `ArrayList` statique n'est plus la source principale des données.

Maintenant :

- Room stocke les livres
- `ExecutorService` exécute les opérations en arrière-plan

- `runOnUiThread()` met à jour l'écran
- `notifyDataSetChanged()` rafraîchit la liste

Étape 7 : Adapter `AddEditActivity.java`

Ouvrir :

```
1 | AddEditActivity.java
```

Vérifier que le mode modification ne dépend plus obligatoirement de la position.

Utiliser ce code :

```
1 import android.content.Intent;
2 import android.os.Bundle;
3 import android.text.TextUtils;
4 import android.widget.Button;
5 import android.widget.EditText;
6 import android.widget.Switch;
7 import android.widget.TextView;
8
9 import androidx.appcompat.app.ActionBar;
10 import androidx.appcompat.app.AppCompatActivity;
11
12 public class AddEditActivity extends AppCompatActivity {
13
14     public static final String EXTRA_MODE = "MODE";
15     public static final String EXTRA_LIVRE = "LIVRE";
16
17     public static final String MODE_ADD = "ADD";
18     public static final String MODE_EDIT = "EDIT";
19
20     private EditText etTitre;
21     private EditText etAuteur;
22     private EditText etIsbn;
23     private Switch switchDisponible;
24     private Button btnEnregistrer;
25     private TextView tvTitreFormulaire;
26
27     private String mode;
28     private Livre livreAModifier;
29
30     @Override
31     protected void onCreate(Bundle savedInstanceState) {
32         super.onCreate(savedInstanceState);
33         setContentView(R.layout.activity_add_edit);
34
35         ActionBar actionBar = getSupportActionBar();
36         if (actionBar != null) {
37             actionBar.setDisplayHomeAsUpEnabled(true);
38         }
39
40         tvTitreFormulaire = findViewById(R.id.tvTitreFormulaire);
41         etTitre = findViewById(R.id.etTitre);
42         etAuteur = findViewById(R.id.etAuteur);
43         etIsbn = findViewById(R.id.etIsbn);
44         switchDisponible = findViewById(R.id.switchDisponible);
45         btnEnregistrer = findViewById(R.id.btnEnregistrer);
46
47         Intent intent = getIntent();
48         mode = intent.getStringExtra(EXTRA_MODE);
49     }
50 }
```

```

50         if (MODE_EDIT.equals(mode)) {
51             tvTitreFormulaire.setText("Modifier le livre");
52
53             livreAModifier = (Livre)
intent.getSerializableExtra(EXTRA_LIVRE);
54
55             if (livreAModifier != null) {
56                 etTitre.setText(livreAModifier.getTitre());
57                 etAuteur.setText(livreAModifier.getAuteur());
58                 etIsbn.setText(livreAModifier.getIsbn());
59
switchDisponible.setChecked(livreAModifier.isDisponible());
60             }
61
62         } else {
63             mode = MODE_ADD;
64             tvTitreFormulaire.setText("Ajouter un livre");
65         }
66
67         btnEnregistrer.setOnClickListener(v -> enregistrerLivre());
68     }
69
private void enregistrerLivre() {
70     String titre = etTitre.getText().toString().trim();
71     String auteur = etAuteur.getText().toString().trim();
72     String isbn = etIsbn.getText().toString().trim();
73     boolean disponible = switchDisponible.isChecked();
74
75     if (!validerFormulaire(titre, auteur, isbn)) {
76         return;
77     }
78
79     Livre livre;
80
81     if (MODE_EDIT.equals(mode) && livreAModifier != null) {
82         livre = new Livre(
83             livreAModifier.getId(),
84             titre,
85             auteur,
86             isbn,
87             disponible
88         );
89     } else {
90         livre = new Livre(
91             0,
92             titre,
93             auteur,
94             isbn,
95             disponible
96         );
97     }
98 }
99
100     Intent resultIntent = new Intent();
101     resultIntent.putExtra(EXTRA_MODE, mode);
102     resultIntent.putExtra(EXTRA_LIVRE, livre);
103
104     setResult(RESULT_OK, resultIntent);
105     finish();
106 }
107
private boolean validerFormulaire(String titre, String auteur,
String isbn) {
108     boolean formulaireValide = true;
109
110     if (TextUtils.isEmpty(titre)) {
111         etTitre.setError("Le titre est obligatoire");
112         formulaireValide = false;
113     }

```

```

114     }
115
116     if (TextUtils.isEmpty(auteur)) {
117         etAuteur.setError("L'auteur est obligatoire");
118         formulaireValide = false;
119     }
120
121     if (!TextUtils.isEmpty(isbn) && isbn.length() < 10) {
122         etIsbn.setError("L'ISBN doit contenir au moins 10
caractères");
123         formulaireValide = false;
124     }
125
126     return formulaireValide;
127 }
128
129 @Override
130 public boolean onSupportNavigateUp() {
131     finish();
132     return true;
133 }
134 }

```

Explication

En mode ajout :

- l'id vaut 0
- Room génère automatiquement le vrai id

En mode modification :

- on garde l'id existant
- Room sait quel livre mettre à jour

Étape 8 : Vérifier AndroidManifest.xml

Room ne demande pas d'ajout dans le Manifest.

Mais les activités doivent être déclarées :

```

1 <activity
2     android:name=".AddEditActivity"
3     android:parentActivityName=".MainActivity" />
4
5 <activity
6     android:name=".DetailActivity"
7     android:parentActivityName=".MainActivity" />

```

Si vos activités sont dans un package `activities`, utiliser :

```

1 <activity
2     android:name=".activities.AddEditActivity"
3     android:parentActivityName=".activities.MainActivity" />
4
5 <activity
6     android:name=".activities.DetailActivity"
7     android:parentActivityName=".activities.MainActivity" />

```

Étape 9 : Tester la persistance

Lancer l'application.

Faire les tests suivants :

1. Ajouter un livre
2. Fermer complètement l'application
3. Relancer l'application
4. Vérifier que le livre est toujours présent
5. Modifier le livre
6. Fermer puis relancer l'application
7. Vérifier que la modification est conservée
8. Supprimer un livre
9. Fermer puis relancer l'application
10. Vérifier que le livre supprimé n'est plus présent

5. Rendu attendu

À la fin du lab, l'application doit permettre :

Écran liste

- afficher les livres depuis la base Room
- ajouter un livre avec le bouton flottant
- modifier un livre avec un clic long
- supprimer un livre avec un clic long

Écran formulaire

- saisir un titre
- saisir un auteur
- saisir un ISBN
- choisir la disponibilité
- enregistrer les données

Persistance

Après fermeture et réouverture de l'application :

- les livres ajoutés doivent rester présents
- les modifications doivent être conservées

- les suppressions doivent être conservées

Description de la capture attendue

La capture doit montrer :

- une liste de livres affichée depuis Room
- un bouton flottant d'ajout
- une boîte de dialogue avec les options “Modifier” et “Supprimer”
- un formulaire d'ajout ou modification
- un livre ajouté toujours présent après redémarrage de l'application

7. Bonus : améliorer le design et aller plus loin

Pour aller plus loin :

- afficher un message quand la liste est vide
- ajouter une icône de suppression
- ajouter une icône de modification
- améliorer la boîte de dialogue d'options
- utiliser `notifyItemInserted()` après ajout
- utiliser `notifyItemChanged()` après modification
- utiliser `notifyItemRemoved()` après suppression
- ajouter un champ “année de publication”
- ajouter une recherche par titre avec une requête `LIKE`
- ajouter des couleurs plus propres pour les badges de disponibilité